ENSET
ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

ADRIA
BUSINESS & TECHNOLOGY

ÉPREUVE TECHNIQUE DE SELECTION — PFE 2027

Plateforme bancaire
à architecture micro-services agentique

Spring Cloud · Spring AI · MCP · GraphQL · Kafka · React · Docker

RAPPORT TECHNIQUE

Architecture micro-services, IA et développement full-stack

Réalisé par :
Fatima EL ABBADI

Organisme :
ADRIA Business & Technology
ENSET Mohammedia

Épreuve pratique — Durée : 3 heures

Repository : github.com/elabbadiaussi/elabbadi-fatima-test-adria-pfe-27

Année universitaire 2026 – 2027 · 30 octobre 2026

Table des matières

Introduction 2
Environnement de travail 2
Partie A — Préparation et organisation du projet 3
A.1 Création du repository GitHub 3
A.2 Projet Maven multi-modules 3
A.3 Stratégie de commits 3
A.4 Séparation des couches 4
Partie B — Conception et architecture 5
B.1 Diagramme de classes 5
B.2 Schéma d'architecture 6
Partie C — Micro-services métier 7
C.1 account-service 7
C.2 transaction-service 9
C.3 Communication OpenFeign et tolérance aux pannes (Resilience4J) 11
Partie D — Micro-services techniques 13
D.1 Eureka Discovery Service 13
D.2 Config Service 13
D.3 Spring Cloud Gateway 14
Partie E — Chatbot agentique 15
E.1 Architecture de l'agent 15
E.2 Tests avec un client REST 15
Partie F — Communication asynchrone avec Kafka 17
F.1 Infrastructure 17
F.2 Publication après persistance réussie 17
F.3 Service consommateur 17
Partie G — Client web React 18
Partie H — IA avec Python : détection de fraude 20
Partie I — DevOps 25
Conclusion 26

Introduction

Ce rapport présente la conception et la réalisation, en trois heures, d'une **plateforme bancaire** fondée sur une architecture micro-services. La plateforme permet de consulter des comptes, d'enregistrer des transactions et des virements, de publier des événements métier et d'interroger les données à l'aide d'un **assistant conversationnel agentique**.

Le travail couvre plusieurs dimensions : architecture micro-services (Spring Cloud), exposition multi-protocoles (REST, GraphQL, MCP), tolérance aux pannes (OpenFeign + Resilience4J), intelligence artificielle (Spring AI, LLM, MCP), communication asynchrone (Kafka), client web (React), data science (détection de fraude en Python) et DevOps (Docker, GitHub Actions).

Pour chaque partie de l'énoncé, le rapport détaille l'**objectif**, les **étapes de réalisation**, les **choix techniques** et les **tests effectués**, illustrés par des captures d'écran.

Environnement de travail

Outil	Version	Usage
Java (JDK)	21 LTS	Langage des micro-services
Apache Maven	3.10	Build multi-modules
Spring Boot	3.5.0	Socle de chaque micro-service
Spring Cloud	2025.0.0	Eureka, Config, Gateway, OpenFeign, Circuit Breaker
Spring AI	1.0.0	ChatClient, MCP Server, MCP Client
Docker	28.1	Kafka, conteneurisation
Node.js / Vite	LTS	Client React
Python	3.10	Détection de fraude (scikit-learn)
IDE	VS Code	Extension Pack for Java, Spring Boot Extension Pack

Partie A — Préparation et organisation du projet

A.1 Création du repository GitHub

1. Création d'un repository nommé `elabbadi-fatima-test-adria-pfe-27` (format imposé nom-prenom-test-adria-pfe-27), sans README ni licence.
2. Création du dossier local du projet, ouverture dans VS Code, puis `git init`, `git branch -M main` et `git remote add origin …`.
3. Premier commit `init: maven parent project` et push vers GitHub. Le repository est rendu public à la fin de l'épreuve.

A.2 Projet Maven multi-modules

Conformément à la consigne « créer un projet Maven incluant un module pour chaque micro-service », la racine est un **projet Maven parent** (`<packaging>pom</packaging>`) qui hérite de `spring-boot-starter-parent`. Il centralise les versions (Java 21, Spring Cloud, Spring AI, springdoc) via `dependencyManagement`. Chaque micro-service est un **module** et un **projet Spring Boot autonome** : classe `@SpringBootApplication`, `pom.xml` avec `spring-boot-maven-plugin`, wrapper `mvnw` et `application.properties`.

```
<packaging>pom</packaging>
<modules>
    <module>discovery-service</module>
    <module>config-service</module>
    <module>gateway-service</module>
    <module>account-service</module>
    <module>transaction-service</module>
    <module>bank-agent</module>
    <module>audit-service</module>
</modules>
```

Module	Port	Responsabilité
discovery-service	8761	Annuaire de services Eureka
config-service	9999	Configuration centralisée (dépôt `config-repo`)
gateway-service	8888	Point d'entrée unique, routage, CORS
account-service	8081	Gestion des comptes — REST, GraphQL, MCP
transaction-service	8082	Transactions et virements — REST, GraphQL, MCP, Feign, Resilience4J, producteur Kafka
bank-agent	8085	Agent IA — Spring AI, client MCP, LLM
audit-service	8086	Consommateur Kafka (journalisation)
web-client	5173	Client web React (Vite)

A.3 Stratégie de commits

Un commit est poussé après chaque étape validée par des tests, à intervalles réguliers, avec des messages au format *conventional commits* :

```
init: maven parent project
docs: add class diagram and architecture schema
feat(account-service): add domain, repository, service and REST API
feat(account-service): expose GraphQL and MCP connectors
feat(transaction-service): add REST, GraphQL, MCP with OpenFeign and Resilience4J
feat: add discovery (Eureka), config server with config-repo and gateway
feat(bank-agent): add AI agent with Spring AI, Groq LLM and MCP tools
feat(kafka): publish TransactionCreated events and add audit-service consumer
feat(web-client): add React client for accounts, transactions, payments and AI chat
```

A.4 Séparation des couches

Chaque micro-service métier respecte la structure imposée :

```
ma.adria.<service>/
├── domain/        entités JPA (Account, Transaction, enums)
├── repository/    interfaces Spring Data JPA
├── service/       logique et règles métier (unique source de vérité)
├── web/rest/      @RestController + gestion des erreurs HTTP
├── web/graphql/   @QueryMapping / @MutationMapping + gestion des erreurs GraphQL
└── mcp/           @Tool exposés via le serveur MCP
```

Principe clé : les contrôleurs REST, GraphQL et les outils MCP appellent tous **la même couche service**. Les règles métier ne sont écrites qu'une seule fois et restent cohérentes quel que soit le canal (web, API ou agent IA).

Partie B — Conception et architecture

B.1 Diagramme de classes

Le domaine s'articule autour de deux agrégats : `Account` (géré par account-service) et `Transaction` (géré par transaction-service). Les deux services ne partagent pas de base de données : `Transaction` référence un compte uniquement par son `accountId`, et la cohérence est assurée par des appels OpenFeign. `TransactionCreatedEvent` matérialise l'événement publié dans Kafka.

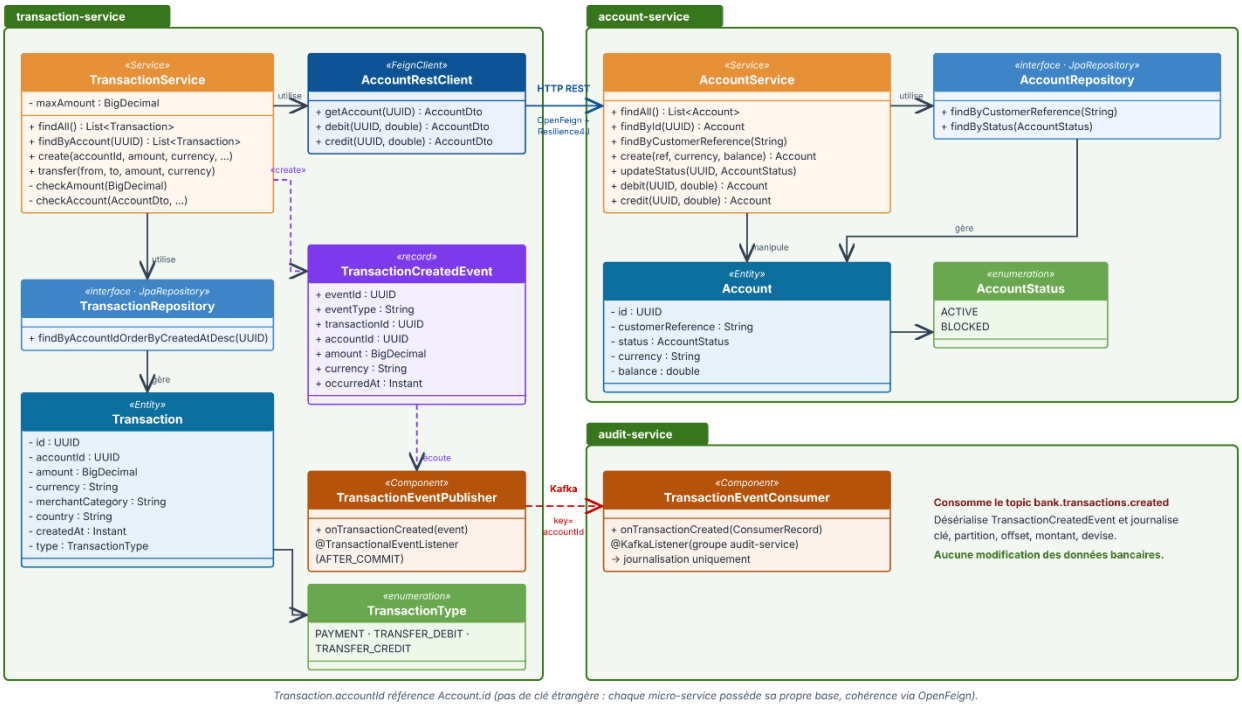


***Figure 1** – Diagramme de classes : domaine, services, client Feign et événements*

B.2 Schéma d'architecture

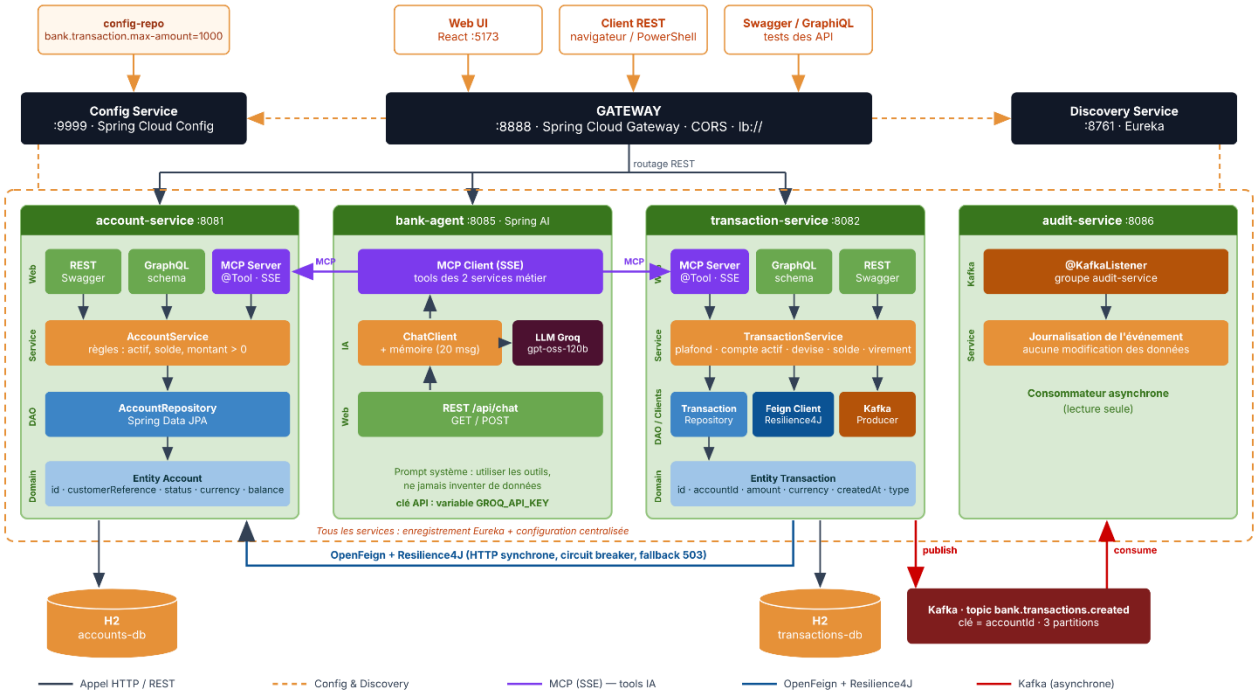


***Figure 2** – Schéma d'architecture de la plateforme*

Flux principaux :

- Les clients (React, client REST) passent par la **gateway**, qui route vers les services enregistrés dans **Eureka** (URI `lb://...`, avec équilibrage de charge).
- Les services récupèrent leur configuration partagée auprès du **config-service** (`bank.transaction.max-amount`).
- **transaction-service** consulte account-service via **OpenFeign**, protégé par un **circuit breaker Resilience4J**.
- Après chaque transaction persistée, un événement est publié dans le topic Kafka `bank.transactions.created` et consommé par **audit-service**.
- L'**agent IA** (Spring AI) utilise un LLM et appelle les outils exposés par les **serveurs MCP** des services métier.

Partie C — Micro-services métier

C.1 account-service

Étape 1 — Génération du projet

Le projet est généré avec Spring Initializr (VS Code) : Group `ma.adria`, Artifact `account-service`, Jar, Java 21, dépendances Spring Web, Spring Data JPA, H2, Lombok, Validation, Actuator, Eureka Discovery Client et Config Client. Le `pom.xml` est ensuite rattaché au parent `bank-platform`.

Étape 2 — Domaine, persistance et règles métier

L'entité `Account` est persistée dans une base H2 en mémoire. Un `CommandLineRunner` initialise quatre comptes de test (trois en EUR, dont un **bloqué**, et un en MAD). `AccountService` porte les règles suivantes :

- débit et crédit autorisés uniquement sur un compte **ACTIVE** ;
- montant strictement positif ;
- **solde suffisant** obligatoire pour un débit ;
- création de compte avec une devise ISO sur 3 lettres et un solde initial positif ou nul.

```
public Account debit(UUID id, double amount) {
    if (amount <= 0) throw new IllegalArgumentException("Le montant doit etre strictement positif");
    Account account = findById(id);
    if (account.getStatus() != AccountStatus.ACTIVE)
        throw new IllegalStateException("Le compte " + id + " n'est pas actif");
    if (account.getBalance() < amount)
        throw new IllegalStateException("Solde insuffisant sur le compte " + id);
    account.setBalance(account.getBalance() - amount);
    return accountRepository.save(account);
}
```

Étape 3 — Connecteur REST

Méthode	Endpoint	Description
GET	`/api/accounts`	Liste des comptes
GET	`/api/accounts/{id}`	Détail d'un compte
GET	`/api/accounts/customer/{ref}`	Comptes d'un client
POST	`/api/accounts`	Création
PATCH	`/api/accounts/{id}/status`	Blocage / activation
POST	`/api/accounts/{id}/debit`	Débit
POST	`/api/accounts/{id}/credit`	Crédit

Les exceptions sont converties en réponses JSON homogènes par un `@RestControllerAdvice` (404 pour un compte introuvable, 400 pour une règle métier violée). L'API est documentée avec **springdoc-openapi** (Swagger UI).



Figure 3 – *Test REST : GET /api/accounts renvoie les 4 comptes initialisés*

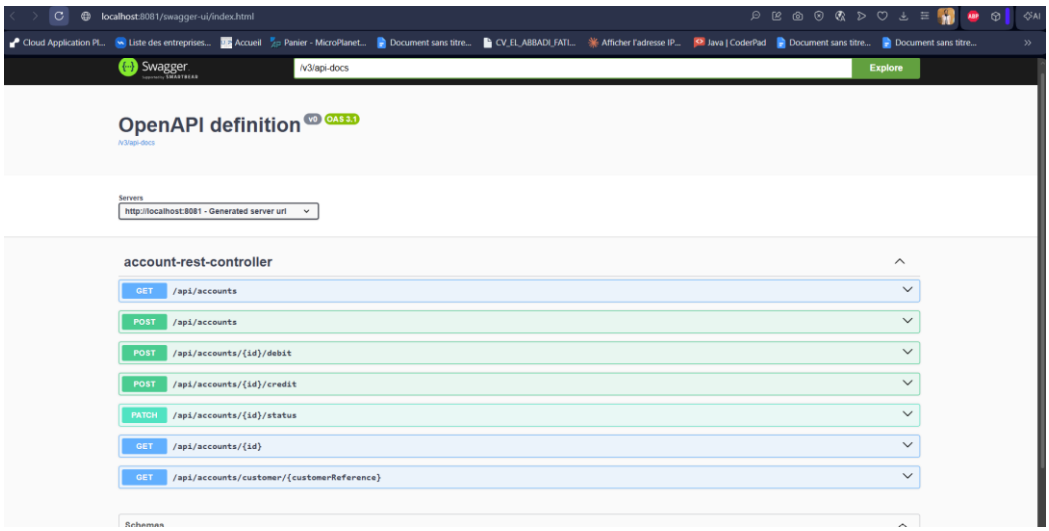


Figure 4 – *Documentation Swagger UI de account-service*

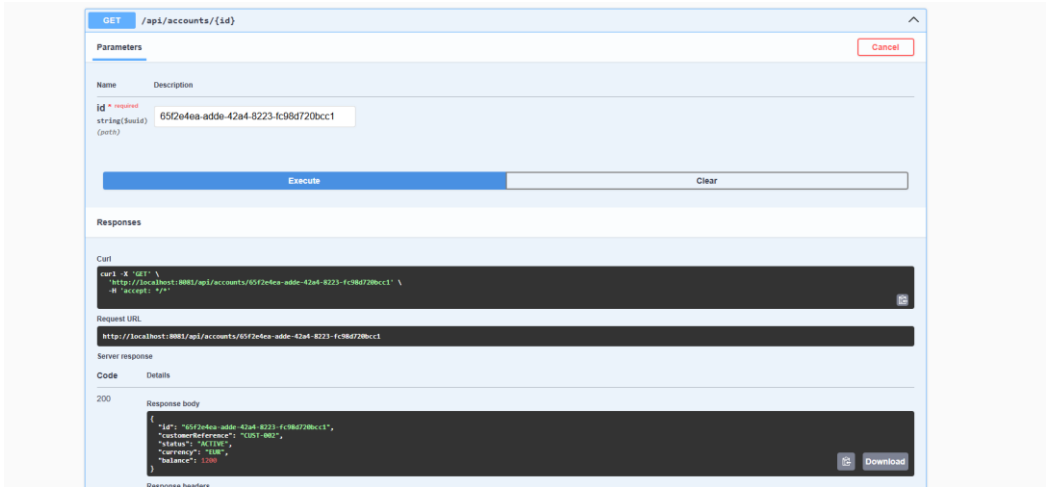


Figure 5 – *Swagger : GET /api/accounts/{id} → 200 OK*

Étape 4 — Connecteur GraphQL

Un schéma `schema.graphqls` déclare le type `Account`, l'enum `AccountStatus`, les requêtes (`accounts`, `accountById`, `accountsByCustomer`) et les mutations (`createAccount`, `updateAccountStatus`, `debitAccount`, `creditAccount`). Le contrôleur `AccountGraphQlController` délègue à `AccountService`, et un `@GraphQlExceptionHandler` traduit les erreurs métier en erreurs GraphQL typées.

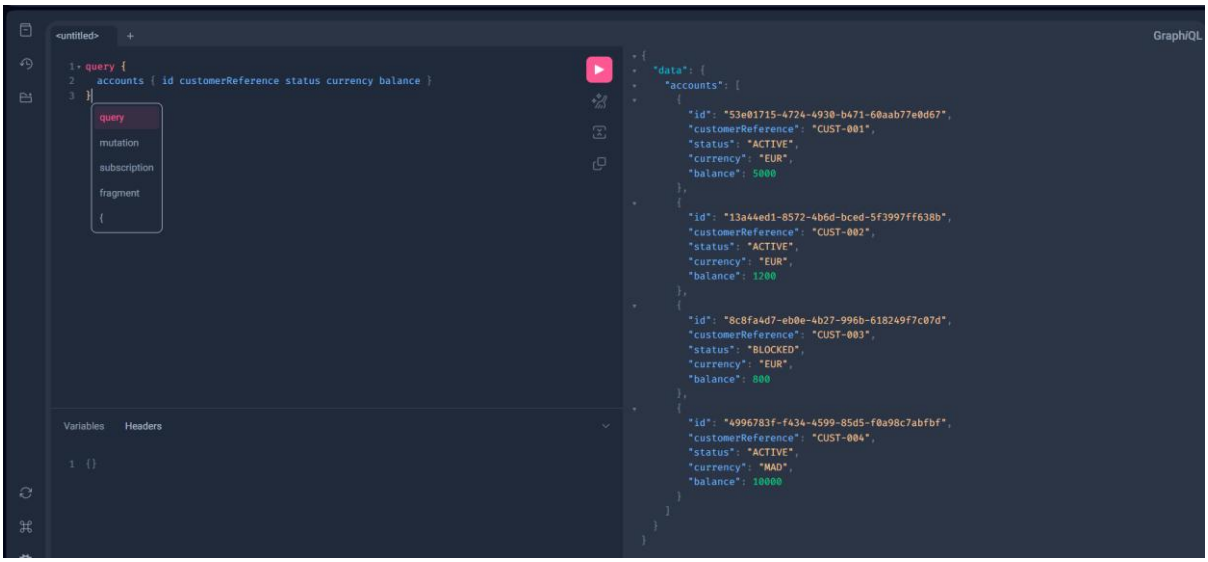


Figure 6 – *GraphiQL : requête accounts*

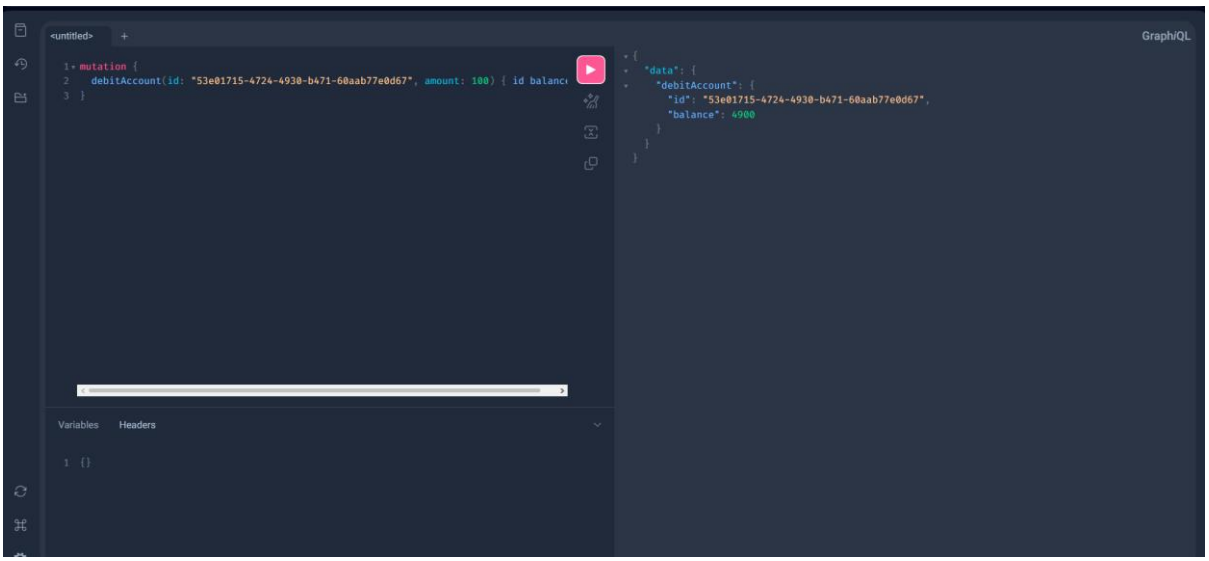


***Figure 7** – GraphiQL : mutation debitAccount, le solde passe de 5000 à 4900*

Étape 5 — Connecteur MCP

Le serveur MCP est fourni par `spring-ai-starter-mcp-server-webmvc` (transport SSE, endpoint `/sse`). Les méthodes de `AccountMcpTools` annotées `@Tool` sont enregistrées via un `MethodToolCallbackProvider`. Six outils sont exposés : `listAccounts`, `getAccount`, `getAccountsByCustomer`, `createAccount`, `blockAccount` et `activateAccount`.

```
@Tool(name = "getAccountsByCustomer",
      description = "Retourne les comptes d'un client a partir de sa reference (ex: CUST-001)")
public List<Account> getAccountsByCustomer(@ToolParam(description = "Reference du client") String
ref) {
    return accountService.findByCustomerReference(ref);
}
```

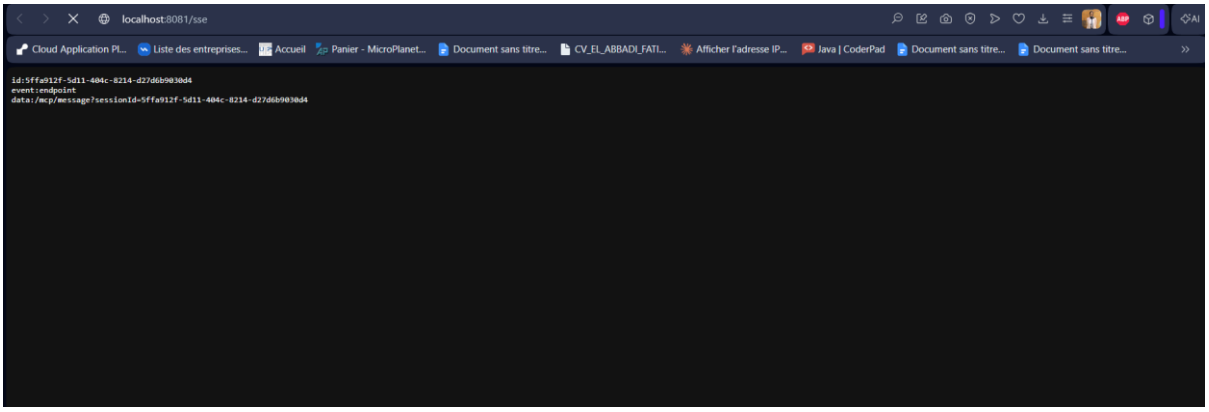


***Figure 8** – Endpoint MCP SSE actif : /sse renvoie event:endpoint et un sessionId*

C.2 transaction-service

Étape 1 — Modèle

L'entité `Transaction` reprend le modèle minimal (id, accountId, amount en `BigDecimal`, currency, merchantCategory, country, createdAt en `Instant`), complété par un champ `type` (PAYMENT, TRANSFER_DEBIT, TRANSFER_CREDIT) qui distingue les paiements des deux jambes d'un virement.

Étape 2 — Règles métier

Règle	Implémentation
Montant strictement positif et ≤ plafond configuré	`checkAmount()` compare au paramètre `bank.transaction.max-amount` (config-service)

Règle	Implémentation
Le compte doit exister et être actif	Appel Feign `getAccount()` : 404 traduit en erreur métier, statut vérifié
Devise identique à celle du compte	`checkAccount()` compare `currency` et `account.currency()`
Consultation du solde avant la transaction	Le solde est lu avant toute écriture et doit couvrir le montant
Mise à jour du solde après enregistrement	`save()` de la transaction puis `accountRestClient.debit()`
Virement compte à compte	`transfer()` : deux transactions, débit puis crédit, avec compensation si le crédit échoue

```
@Transactional
public Transaction create(UUID accountId, BigDecimal amount, String currency, String category, String
country) {
    checkAmount(amount);                                          // > 0 et <= plafond
    AccountDto account = accountRestClient.getAccount(accountId); // existence (OpenFeign)
    checkAccount(account, currency, amount);                      // ACTIVE, devise, solde
    Transaction t = transactionRepository.save(...);              // enregistrement
    accountRestClient.debit(accountId, amount.doubleValue());     // mise a jour du solde
    publish(t);                                                   // evenement Kafka (apres commit)
    return t;
}
```

Étape 3 — Connecteurs REST, GraphQL et MCP

- REST : `GET /api/transactions`, `GET /api/transactions/{id}`, `GET /api/transactions/account/{accountId}`, `POST /api/transactions`, `POST /api/transactions/transfer`, `GET /api/transactions/config/max-amount`.
- GraphQL : `transactions`, `transactionById`, `transactionsByAccount`, `createTransaction`, `transfer`.
- MCP : `listTransactions`, `getTransactionsByAccount`, `createPayment`, `transfer`, `getMaxTransactionAmount`.

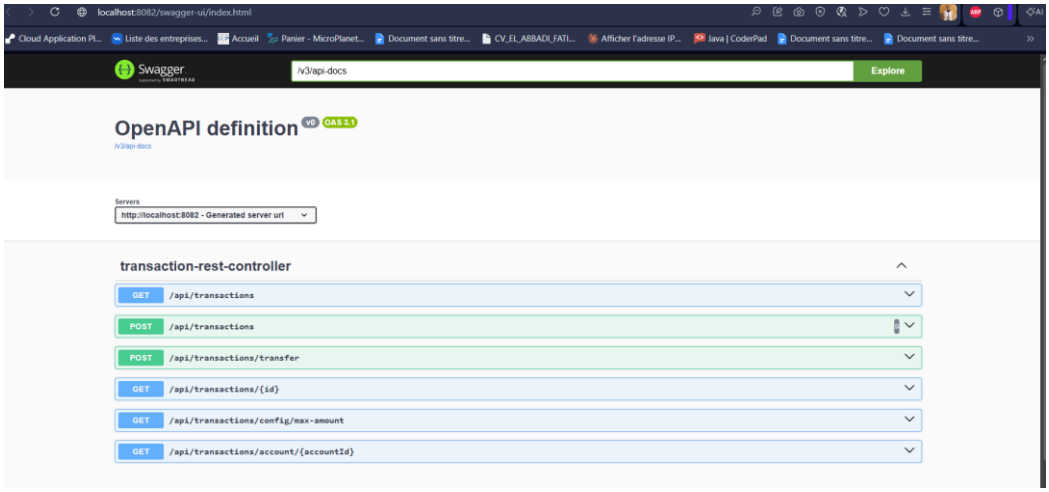


Figure 9 – *Swagger UI de transaction-service*

Étape 4 — Tests

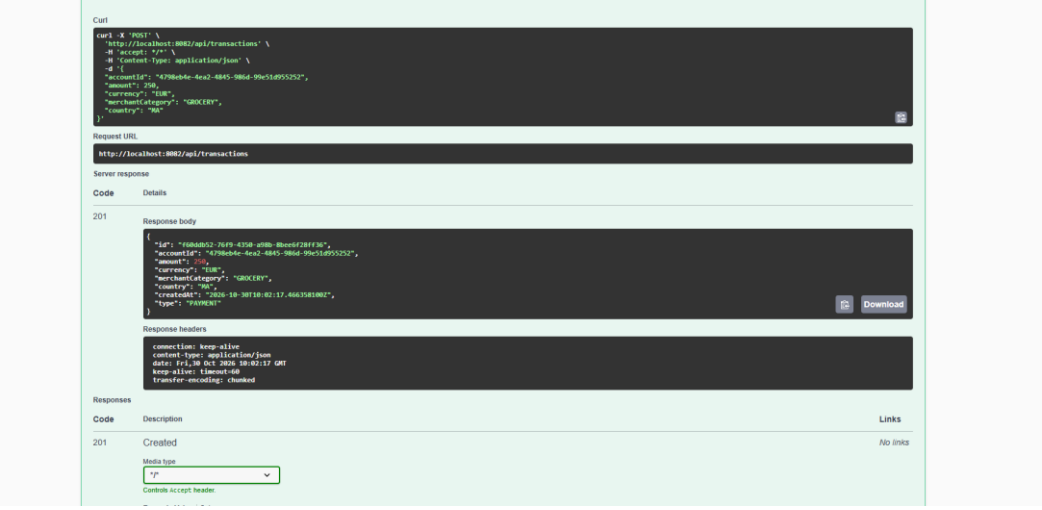


Figure 10 – *Paiement de 250 EUR sur CUST-001 → 201 Created*



Figure 11 – *Vérification : le solde de CUST-001 passe de 5000 à 4750*

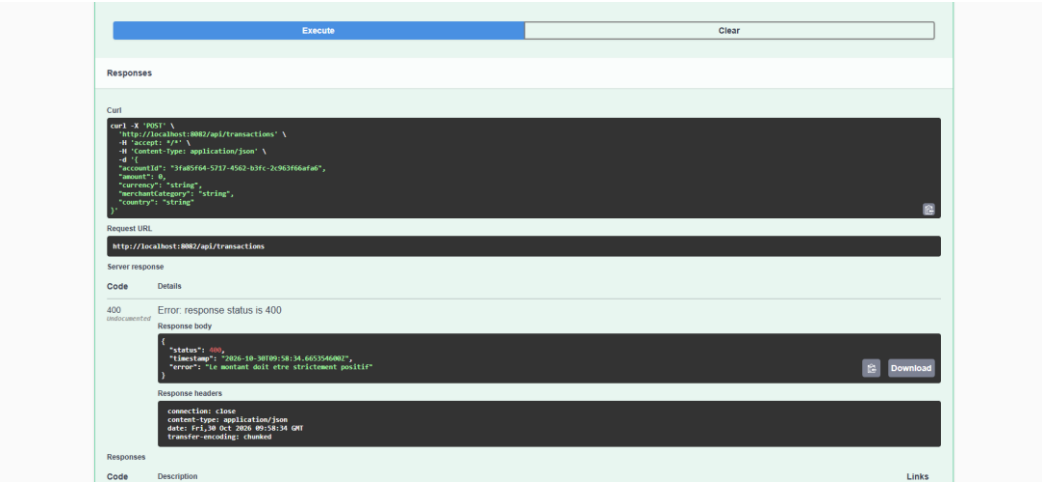


Figure 12 – *Règle métier : un montant nul est refusé (400)*

C.3 Communication OpenFeign et tolérance aux pannes (Resilience4J)

Client OpenFeign

```
@FeignClient(name = "account-service", url = "${account-service.url:}",
             fallbackFactory = AccountRestClientFallbackFactory.class)
public interface AccountRestClient {
    @GetMapping("/api/accounts/{id}")         AccountDto getAccount(@PathVariable UUID id);
    @PostMapping("/api/accounts/{id}/debit")  AccountDto debit(@PathVariable UUID id, @RequestParam
double amount);
    @PostMapping("/api/accounts/{id}/credit") AccountDto credit(@PathVariable UUID id, @RequestParam
double amount);
}
```

Lorsque `account-service.url` est vide (valeur fournie par le config-service), Feign résout `account-service` **via Eureka** avec équilibrage de charge.

Circuit breaker Resilience4J

- Activation : `spring.cloud.openfeign.circuitbreaker.enabled=true`.
- Configuration : fenêtre glissante de 10 appels, minimum de 5 appels, seuil d'échec de 50 %, ouverture pendant 10 s, 2 appels de test en demi-ouverture, time-limiter de 5 s.
- La **FallbackFactory** distingue les **erreurs métier** renvoyées par account-service (404 compte inexistant, 400 compte bloqué ou solde insuffisant), restituées clairement, et les **pannes techniques** (service arrêté, timeout, circuit ouvert), qui produisent une réponse **503 Service Unavailable**.
- Supervision : `/actuator/health` et `/actuator/circuitbreakers`.

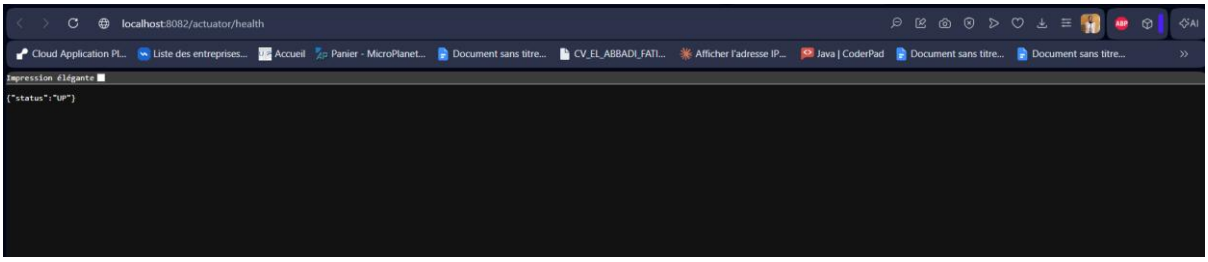


Figure 13 – *Actuator health de transaction-service*

Partie D — Micro-services techniques

D.1 Eureka Discovery Service

`discovery-service` est un serveur Eureka (`@EnableEurekaServer`, port 8761, `register-with-eureka=false`). Tous les autres services s'y enregistrent avec `prefer-ip-address=true`.

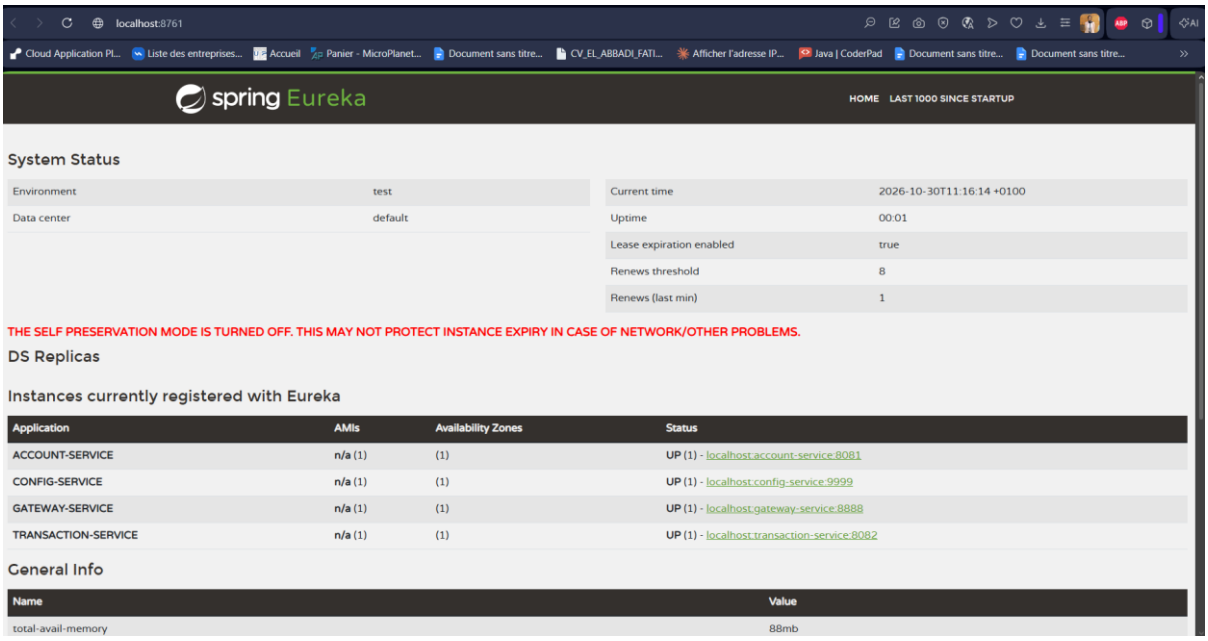


Figure 14 – *Console Eureka : ACCOUNT, CONFIG, GATEWAY et TRANSACTION-SERVICE sont UP*

D.2 Config Service

`config-service` (`@EnableConfigServer`, port 9999) utilise le profil **native** sur le dossier `config-repo/`. Le paramètre demandé est partagé dans `config-repo/application.properties`, visible par tous les services :

```
# config-repo/application.properties
bank.transaction.max-amount=1000

# config-repo/transaction-service.properties
account-service.url=          # vide => resolution via Eureka
```

Les clients importent la configuration au démarrage avec `spring.config.import=optional:configserver:http://localhost:9999`.



Figure 15 – *Config server : configuration servie à transaction-service*

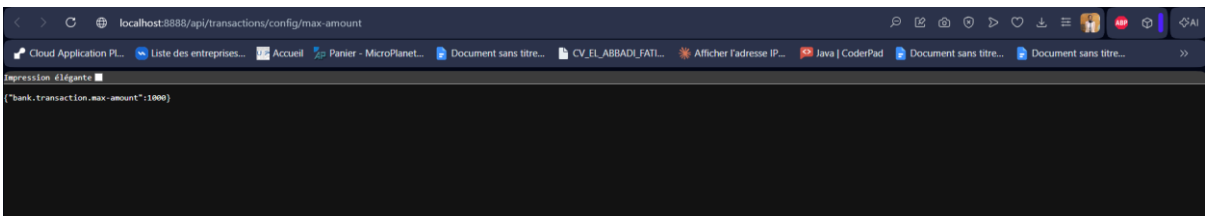


Figure 16 – *Le paramètre max-amount = 1000 est bien reçu par transaction-service (via la gateway)*

D.3 Spring Cloud Gateway

`gateway-service` (port 8888) déclare ses routes en Java avec un `RouteLocator`. Les URIs `lb://` délèguent la résolution à Eureka, et un `CorsWebFilter` global autorise le client web.

Route	Destination
`/api/accounts/**`	`lb://account-service`
`/api/transactions/**`	`lb://transaction-service`
`/api/chat/**`	`lb://bank-agent`
`/account-service/**` (StripPrefix=1)	GraphQL et Swagger de account-service
`/transaction-service/**` (StripPrefix=1)	GraphQL et Swagger de transaction-service



Figure 17 – *Gateway → account-service : GET /api/accounts*

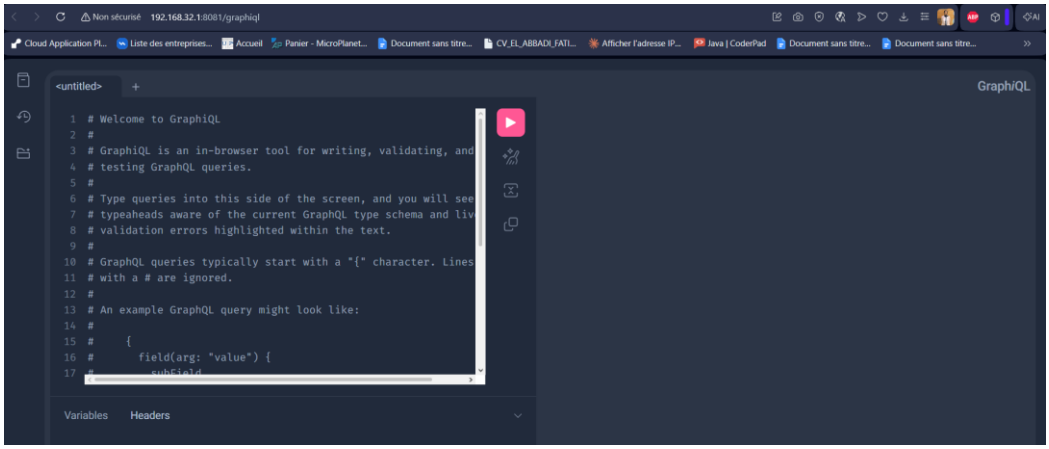


Figure 18 – *GraphiQL atteint via la gateway (/account-service/graphiql)*

Partie E — Chatbot agentique

E.1 Architecture de l'agent

Le module `bank-agent` (port 8085) est construit avec **Spring AI 1.0** :

- **LLM** : Groq (API compatible OpenAI) avec le modèle `openai/gpt-oss-120b`, qui gère bien l'appel d'outils. La clé est fournie par la variable d'environnement `GROQ_API_KEY` et n'est jamais commitée.
- **MCP Client** (transport SSE) connecté aux serveurs MCP d'account-service et de transaction-service. Les outils distants sont injectés dans le `ChatClient` via `SyncMcpToolCallbackProvider`.
- **Mémoire de conversation** : `MessageChatMemoryAdvisor` (fenêtre de 20 messages par `conversationId`).
- **Prompt système** : l'agent doit toujours utiliser les outils, ne jamais inventer de données, résoudre la référence client en UUID et expliquer les erreurs métier.
- **API REST** : `GET/POST /api/chat`, exposée via la gateway.

```
@Bean
public ChatClient chatClient(ChatClient.Builder builder, List<McpSyncClient> mcpClients, ChatMemory
memory) {
    return builder.defaultSystem(SYSTEM_PROMPT)
            .defaultToolCallbacks(new SyncMcpToolCallbackProvider(mcpClients))
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(memory).build())
            .build();
}
```

E.2 Tests avec un client REST

Les tests sont faits avec un client REST (navigateur, PowerShell `Invoke-RestMethod` et client React) via la gateway. Pour chaque demande, le LLM choisit et enchaîne les outils MCP nécessaires.

Requête utilisateur	Outils MCP appelés	Résultat
Liste tous les comptes	`listAccounts`	Tableau des 4 comptes
Solde du client CUST-001 ?	`getAccountsByCustomer`	5 000,00 EUR
Paiement de 200 EUR chez GROCERY pour CUST-001	`getAccountsByCustomer` → `createPayment`	Transaction créée, solde 4 800 EUR
Virement de 50 EUR de CUST-001 vers CUST-002	`getAccountsByCustomer` ×2 → `transfer`	Débit et crédit créés
Paye 5000 EUR avec CUST-001	`getAccountsByCustomer`	Refus expliqué (solde et plafond)

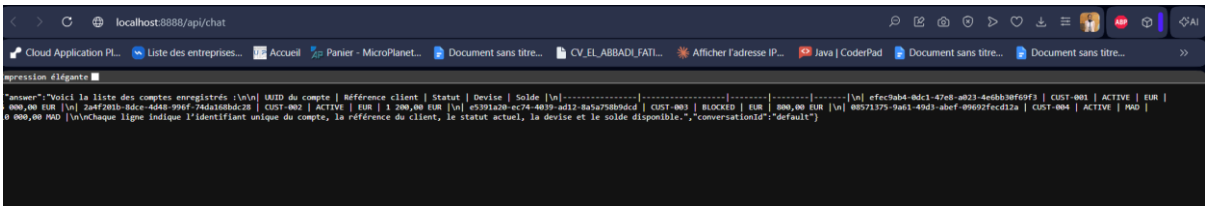


Figure 19 – *Agent : « Liste tous les comptes »*



Figure 20 – *Agent : solde du client CUST-001*

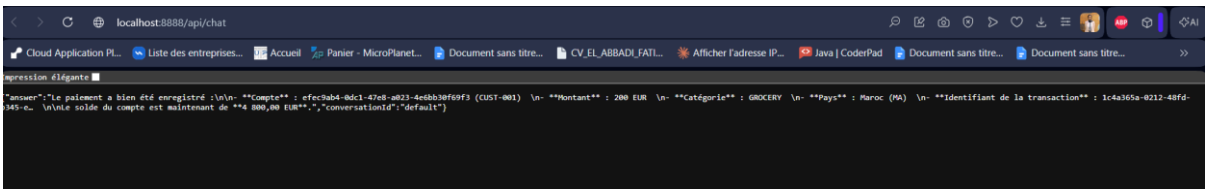


Figure 21 – *Agent : paiement de 200 EUR exécuté*



Figure 22 – *Agent : virement de 50 EUR de CUST-001 vers CUST-002*



Figure 23 – *Agent : refus motivé d'un paiement de 5000 EUR*

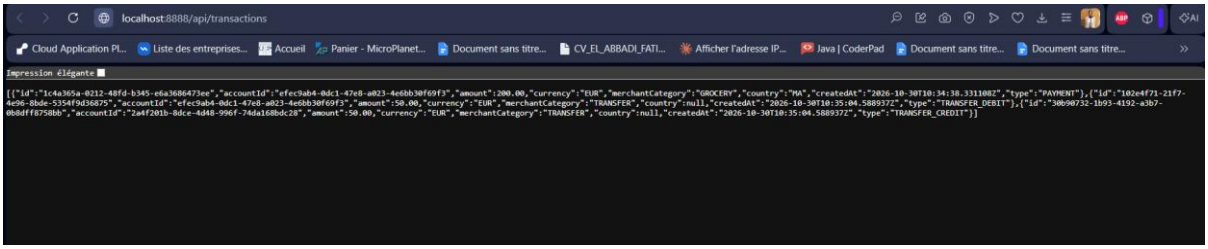


Figure 24 – *Vérification : transactions créées par l'agent (GET /api/transactions)*

Partie F — Communication asynchrone avec Kafka

F.1 Infrastructure

Kafka 3.9 (mode **KRaft**, sans ZooKeeper) et Kafka UI sont lancés avec `docker compose -f docker-compose.kafka.yml up -d`. Le topic `bank.transactions.created` (3 partitions) est créé par un bean `NewTopic`.

F.2 Publication après persistance réussie

`TransactionService` publie un événement applicatif. `TransactionEventPublisher` l'envoie dans Kafka grâce à `@TransactionalEventListener(phase = AFTER_COMMIT)`, ce qui garantit que l'événement n'est publié **que si la transaction a bien été enregistrée en base** (aucun événement si la transaction est annulée).

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onTransactionCreated(TransactionCreatedEvent event) {
    String key = event.accountId().toString();          // cle = accountId
    kafkaTemplate.send(topic, key, objectMapper.writeValueAsString(event));
}
```

Pourquoi accountId comme clé ? Kafka envoie tous les messages de même clé dans la même partition. Les événements d'un même compte sont donc consommés **dans l'ordre**, ce qui est essentiel pour des opérations bancaires.

```
{
  "eventId": "UUID", "eventType": "TransactionCreated", "transactionId": "UUID",
  "accountId": "UUID", "amount": 250.00, "currency": "EUR",
  "occurredAt": "2026-10-30T10:30:00Z"
}
```

F.3 Service consommateur

`audit-service` (Spring Boot, port 8086) consomme le topic avec `@KafkaListener` (groupe `audit-service`, `auto-offset-reset=earliest`). Il **journalise** chaque événement (clé, partition, offset, montant) **sans modifier les données bancaires**.

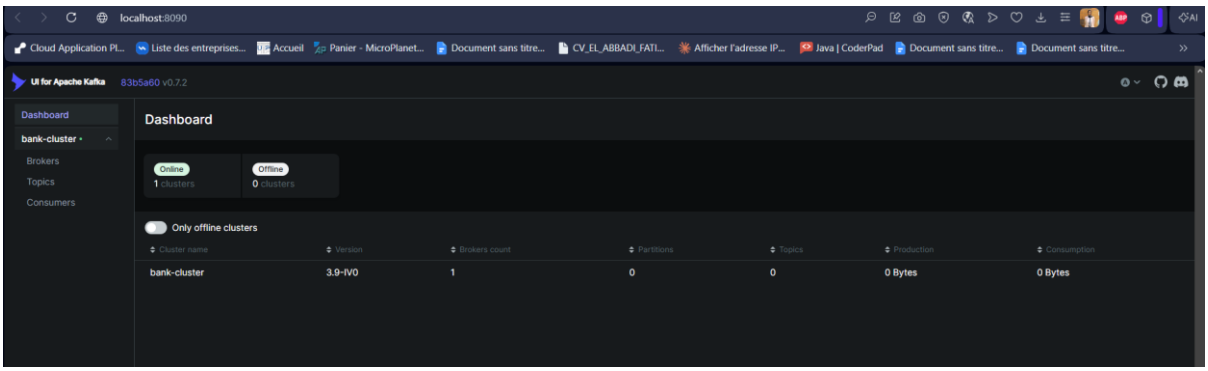


Figure 25 – *Kafka UI : cluster bank-cluster en ligne*

Partie G — Client web React

Le choix s'est porté sur **React** (Vite), plus rapide à mettre en place. L'application consomme toutes les API via la **gateway** (`http://localhost:8888`) et comporte quatre onglets :

- **Comptes** : liste avec statut, devise et solde, et bouton pour bloquer ou activer un compte ;
- **Transactions** : historique trié, montants crédit/débit colorés ;
- **Paiement / Virement** : formulaires avec devise pré-remplie selon le compte et affichage des erreurs métier renvoyées par l'API ;
- **Assistant IA** : interface de chat connectée à `/api/chat` avec mémoire de conversation.

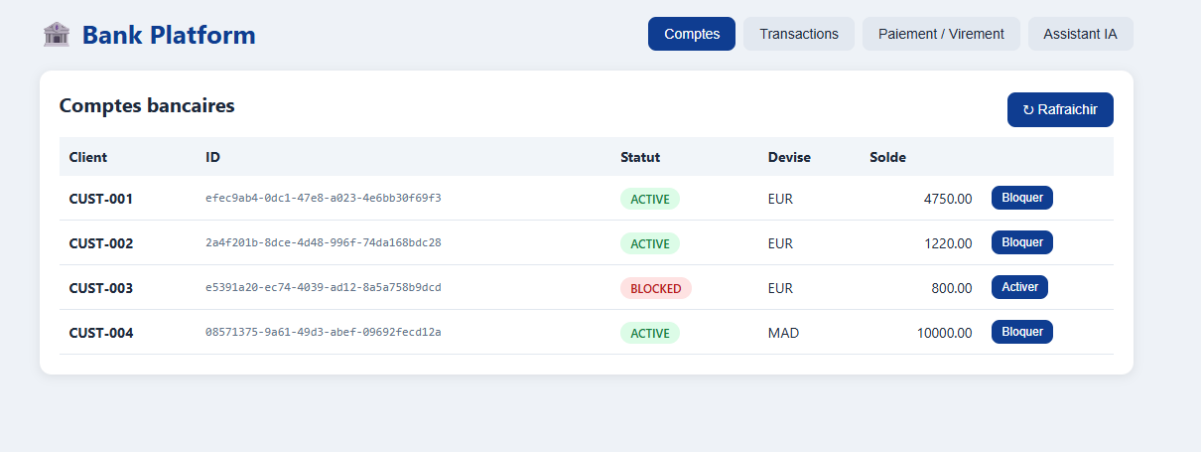


Figure 26 – *React : onglet Comptes*

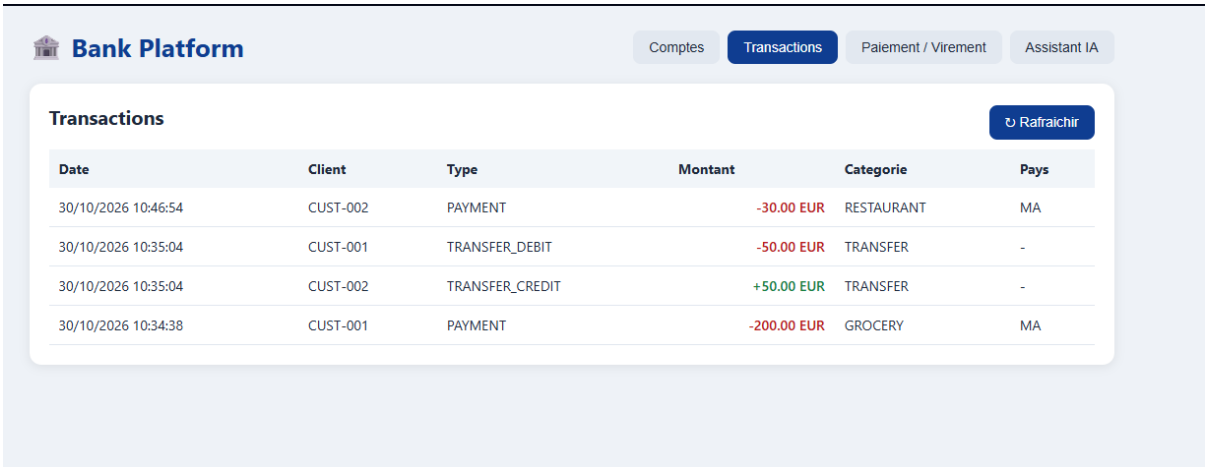


Figure 27 – *React : onglet Transactions*

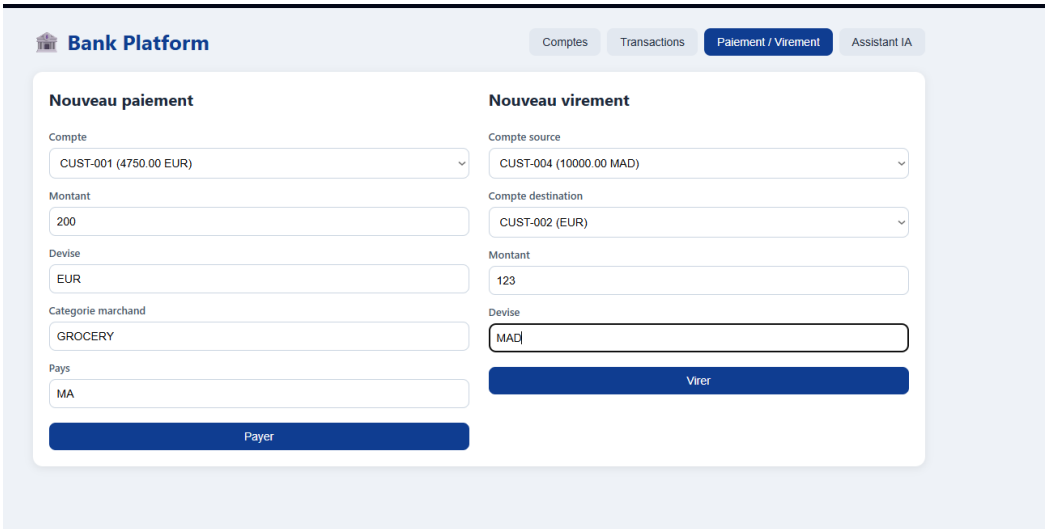


Figure 28 – *React : formulaires de paiement et de virement*

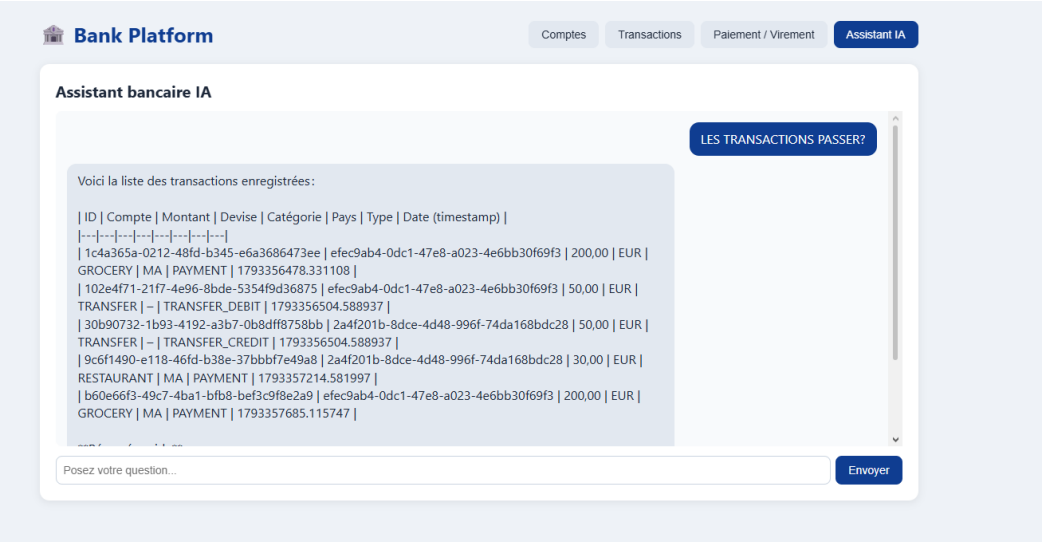


Figure 29 – *React : assistant IA*

Partie H — IA avec Python : détection de fraude

H.1 Objectif

Construire un modèle de **classification binaire** qui prédit si une transaction est frauduleuse (is_fraud = 1) à partir du fichier bank_transactions.csv, en respectant deux contraintes : le prétraitement est ajusté **uniquement sur l'entraînement**, et l'ensemble de **test** ne sert ni à entraîner le modèle ni à choisir le seuil de décision.

H.2 Étape 1 : chargement des données

Le jeu de données contient **5 000 transactions** et **9 colonnes** :

- **Identifiants** : transaction_id, account_id
- **Variables numériques** : amount, hour, transactions_last_24h, average_amount_last_7d
- **Variables catégorielles** : country, merchant_category
- **Cible** : is_fraud (0 ou 1)

```
============================================================
1. CHARGEMENT DES DONNEES
============================================================
Dimensions : 5000 lignes x 9 colonnes
  transaction_id account_id  amount  hour country merchant_category  transactions_last_24h  average_amount_last_7d  is_fraud
0       TX000001    ACC0090  383.72     4      FR         ECOMMERCE                      6                  217.44         0
1       TX000002    ACC0774  516.48     0      BE         ECOMMERCE                      1                  275.06         0
2       TX000003    ACC0655  151.95    15      DE           GROCERY                      1                  100.82         0
3       TX000004    ACC0439   34.56    16      FR           GROCERY                      4                   33.88         0
4       TX000005    ACC0434   93.14    18      FR         TRANSPORT                      0                  276.66         0

Types :
transaction_id               str
account_id                   str
amount                   float64
hour                       int64
country                      str
merchant_category            str
transactions_last_24h      int64
average_amount_last_7d   float64
```

H.3 Étape 2 : proportion de fraudes et valeurs manquantes

Classe	Effectif	Proportion
0 – légitime	4 743	94,86 %
1 – fraude	257	**5,14 %**

Les classes sont **fortement déséquilibrées** : environ 1 fraude pour 18 transactions légitimes.

Colonne	Valeurs manquantes	%
country	66	1,32 %
merchant_category	84	1,68 %
average_amount_last_7d	75	1,50 %
autres colonnes	0	0 %

Les valeurs manquantes sont peu nombreuses et sont traitées par **imputation** dans le pipeline.

```
============================================================
2. PROPORTION DE FRAUDES ET VALEURS MANQUANTES
============================================================
          effectif  proportion (%)
is_fraud
0             4743           94.86
1              257            5.14

=> Taux de fraude : 5.14 % (classes desequilibrees)

Valeurs manquantes par colonne :
                       manquants  pourcentage (%)
transaction_id                 0            0.00
account_id                     0            0.00
amount                         0            0.00
hour                           0            0.00
country                       66            1.32
merchant_category             84            1.68
transactions_last_24h          0            0.00
average_amount_last_7d        75            1.50
is_fraud                       0            0.00

------------------------------------------------------------
```

H.4 Étapes 3 et 4 : séparation stratifiée et exclusion des identifiants

- transaction_id et account_id sont **exclus** des variables explicatives. Ce sont des identifiants arbitraires : les garder ferait apprendre au modèle des comptes précis au lieu de comportements, ce qui provoque du surapprentissage et ne se généralise pas aux nouveaux comptes.
- La séparation est **stratifiée** sur is_fraud pour conserver le même taux de fraude dans chaque sous-ensemble :

Ensemble	Lignes	Taux de fraude	Usage
Entraînement	3 000	5,17 %	Apprentissage
Validation	1 000	5,10 %	Choix du modèle et du seuil
Test	1 000	5,10 %	Évaluation finale uniquement

H.5 Étape 5 : pipeline de prétraitement

Le prétraitement est encapsulé dans un Pipeline scikit-learn avec un ColumnTransformer. Il est donc **ajusté (fit) uniquement sur les données d'entraînement** et seulement *appliqué* (transform) à la validation et au test, ce qui évite toute fuite de données.

Type de variable	Imputation	Transformation
Numériques	médiane	StandardScaler (normalisation)
Catégorielles	modalité la plus fréquente	OneHotEncoder(handle_unknown="ignore")

H.6 Étape 6 : entraînement de modèles simples

Deux modèles ont été comparés, tous deux avec une **pondération des classes** (class_weight="balanced") pour compenser le déséquilibre :

Modèle	Average Precision (validation)	Seuil optimal F1 (validation)	F1 (validation)
Régression logistique	**0,166**	0,711	0,266
Random Forest (300 arbres)	0,124	0,265	0,197

La **régression logistique** est retenue (meilleure average precision en validation). Elle est ensuite ré-entraînée sur entraînement + validation, puis évaluée **une seule fois** sur le test.

```
================================================================
5-6. PIPELINE + ENTRAINEMENT (selection sur VALIDATION)
================================================================
LogisticRegression   AP(val) = 0.1658 | seuil optimal F1 (val) = 0.711 | F1(val) = 0.2656
RandomForest         AP(val) = 0.1241 | seuil optimal F1 (val) = 0.265 | F1(val) = 0.1970

=> Modele retenu (meilleure AP en validation) : LogisticRegression
=> Seuil de decision retenu (choisi sur la validation) : 0.711
```

H.7 Étape 7 : évaluation sur le test

Métrique (classe fraude)	**Seuil 0,5**	**Seuil 0,711 (choisi en validation)**
Précision	0,122	0,167
Rappel	**0,627**	0,235
F1-score	0,205	0,195
Average precision	0,189	0,189
Accuracy	0,75	0,90

Matrices de confusion (test : 949 légitimes, 51 fraudes) :

	Seuil 0,5	**Seuil 0,711**
Vrais négatifs (TN)	719	889
Faux positifs (FP, fausses alertes)	230	60
Faux négatifs (FN, fraudes non détectées)	19	39
Vrais positifs (TP)	32	12

Interprétation :

- L'**average precision de 0,189** est environ **3,7 fois supérieure au hasard** (0,051, soit le taux de fraude) : le modèle apprend un vrai signal, même s'il reste limité.
- Au **seuil 0,5**, le modèle détecte **63 % des fraudes** (32/51), au prix de 230 fausses alertes.
- Au **seuil 0,711**, les fausses alertes tombent à 60, mais seulement 24 % des fraudes sont détectées.
- Les performances restent modestes : avec seulement 257 fraudes et 6 variables, il faudrait plus de données et de variables comportementales (historique du client, appareil, géolocalisation…) pour aller plus loin.

```
======================================================================
7. EVALUATION FINALE SUR LE TEST - LogisticRegression
======================================================================

--- seuil par defaut 0.5 ---
Precision (fraude) : 0.1221
Rappel    (fraude) : 0.6275
F1-score  (fraude) : 0.2045
Average precision  : 0.1886
Matrice de confusion :
[[719 230]
 [ 19  32]]
  TN=719  FP=230 (fausses alertes)  FN=19 (fraudes non detectees)  TP=32
              precision    recall  f1-score   support

    legitime       0.97      0.76      0.85       949
      fraude       0.12      0.63      0.20        51

    accuracy                           0.75      1000
   macro avg       0.55      0.69      0.53      1000
weighted avg       0.93      0.75      0.82      1000


--- seuil choisi en validation 0.711 ---
```

```
--- seuil choisi en validation 0.711 ---
Precision (fraude) : 0.1667
Rappel    (fraude) : 0.2353
F1-score  (fraude) : 0.1951
Average precision  : 0.1886
Matrice de confusion :
[[889  60]
 [ 39  12]]
  TN=889  FP=60 (fausses alertes)  FN=39 (fraudes non detectees)  TP=12
              precision    recall  f1-score   support

    legitime       0.96      0.94      0.95       949
      fraude       0.17      0.24      0.20        51

    accuracy                           0.90      1000
   macro avg       0.56      0.59      0.57      1000
weighted avg       0.92      0.90      0.91      1000
```

H.8 Questions d'analyse

1. Pourquoi l'accuracy seule est-elle insuffisante ?
Avec 94,86 % de transactions légitimes, un modèle qui prédit **toujours « légitime »** obtient 94,86 % d'accuracy tout en détectant **0 fraude**. Nos résultats le confirment : au seuil 0,711, l'accuracy monte à **90 %** alors que le modèle rate 39 fraudes sur 51 ; au seuil 0,5, l'accuracy baisse à 75 % mais le modèle détecte 2,7 fois plus de fraudes. L'accuracy est dominée par la classe majoritaire. Il faut donc regarder les métriques de la **classe fraude** : précision, rappel, F1, average precision (aire sous la courbe précision-rappel) et matrice de confusion.

2. Quel compromis existe entre faux positifs et fraudes non détectées ?

- Un **faux positif** est une transaction légitime bloquée : client mécontent, appels au service client, perte de chiffre d'affaires et d'image.
- Un **faux négatif** est une fraude non détectée : perte financière directe, remboursement du client, risque réglementaire.

Les deux erreurs varient en sens inverse : réduire l'une augmente l'autre. Dans nos résultats, passer du seuil 0,5 au seuil 0,711 fait baisser les fausses alertes de **230 à 60**, mais fait monter les fraudes

manquées de **19 à 39**. Le bon réglage dépend des **coûts métier** : une fraude manquée coûte généralement bien plus cher qu'une vérification supplémentaire, ce qui pousse à privilégier le rappel tout en gardant un volume d'alertes que les analystes peuvent traiter.

3. Quel est l'effet possible d'une diminution du seuil de décision ?
Diminuer le seuil classe plus de transactions comme frauduleuses :

- le **rappel augmente** (plus de fraudes détectées, moins de FN) ;
- la **précision diminue** (plus de fausses alertes, plus de FP) ;
- le F1 et l'accuracy peuvent baisser.

C'est exactement ce qu'on observe en passant de 0,711 à 0,5 : le rappel passe de **23,5 % à 62,7 %**, la précision de 16,7 % à 12,2 %, et les fausses alertes de 60 à 230. Le seuil doit être choisi sur un **jeu de validation** (jamais sur le test), en fonction d'un objectif métier, par exemple un rappel minimal de 80 % ou un nombre maximal d'alertes par jour.

4. Pourquoi privilégier une validation temporelle sur des données bancaires réelles ?

- En production, le modèle prédit **le futur à partir du passé**. Une séparation aléatoire mélange les périodes et fait apprendre au modèle des transactions **postérieures** à celles qu'il évalue, ce qui crée une **fuite d'information** et des scores trop optimistes.
- Les fraudeurs **changent de techniques** dans le temps (*concept drift*). Une validation temporelle (entraînement sur les mois M1 à M6, test sur M7) mesure la capacité réelle du modèle à généraliser à de nouveaux comportements.
- Les variables historiques (transactions_last_24h, average_amount_last_7d) dépendent de l'ordre chronologique. Seule une validation respectant cet ordre est cohérente avec leur calcul.
- Les transactions d'un même compte sont corrélées. Une validation temporelle (éventuellement groupée par compte) évite que le modèle reconnaisse des comptes déjà vus.

Concrètement, on utiliserait TimeSeriesSplit ou un découpage par date, puis un suivi des performances en production pour détecter la dérive et ré-entraîner le modèle.

Partie I — DevOps

Conclusion

Cette épreuve a permis de livrer, en un temps limité, une plateforme bancaire fonctionnelle de bout en bout : deux micro-services métier exposés en **REST, GraphQL et MCP** autour d'une couche service unique, une infrastructure **Spring Cloud** complète (Eureka, Config, Gateway), une communication synchrone résiliente (**OpenFeign + Resilience4J**) et asynchrone (**Kafka**), un **agent IA** capable d'exécuter de vraies opérations bancaires via MCP, et un **client web React**.

Pistes d'amélioration : sécurisation avec **Keycloak** (OAuth2/JWT sur la gateway et les services), bases PostgreSQL persistantes, tests d'intégration (Testcontainers), outbox pattern pour fiabiliser la publication Kafka, observabilité (Prometheus, Grafana, OpenTelemetry) et intégration du modèle de détection de fraude dans le flux de transactions.